DATA INTEGRATION

Data Integration is a technique of integrating the data which resides in different sources. The goal is to provide the users with a holistic view of the data. It can be viewed more as a practice of consolidating data from various disparate sources. This is viewed as one of the most important steps in Data preprocessing. Data Architects are involved in architecting the process which facilitates an automated process of connecting and routing of data from source systems to target systems. This can be achieved by following some of the below techniques:

1. Data Replication: data is replicated in databases for backup purposes.
2. Change Data Capture: data changes are captured and identifies in databases in real-time and are applied to data warehouses and other data storage repositories.
3. Extract, transform and load: this involves extracting the data from different sources, transforming and harmonizing it and loading it into data warehouse or database.
4. Extract, load and transform: this involves loading the data into a big data system and later transforming it.
5. Data Virtualization: this involves virtually combining the data from different systems to create a unified view.
6. Streaming Data Integration: this involves real time Data Integration where multiple streaming system are integrated.

In this post, we will take a basic approach of integrating data using python. This will give you a basic understanding on how we could integrate different datasets while doing data pre-processing.

Integrating Data

Loading the data for "student.csv" and "mark.csv"

```
import pandas as pd
import numpy as np

student = pd.read_csv("student.csv",header=0)
df_merge = pd.merge(student,mark, on = 'Student_id')
```

Challenges of Data Integration

At first glance, the biggest challenge is the technical implementation of integrating data from disparate often incompatible sources. However, a much bigger challenge lies in the entirety of data integration. It has to include the following phases:

Design

- The data integration initiative within a company must be an initiative of business, not IT. There should be a champion who understands the data assets of the enterprise and will be able to lead the discussion about the long-term data integration initiative in order to make it consistent, successful and beneficial.
- Analysis of the requirements (BRS), i.e. why is the data integration being done, what are the objectives and deliverables. From what systems will the data be sourced? Is all the data available to fulfill the requirements? What are the business rules? What is the support model and SLA?
- Analysis of the source systems, i.e. what are the options of extracting the data from the systems (update notification, incremental extracts, full extracts), what is the required/available frequency of the extracts? What is the quality of the data? Are the required data fields populated properly and consistently? Is the documentation available? What are the data volumes being processed? Who is the system owner?
- Any other non-functional requirements such as data processing window, system response time, estimated number of (concurrent) users, data security policy, backup policy.
- What is the support model for the new system? What are the SLA requirements?
- And last but not least, who will be the owner of the system and what is the funding of the maintenance and upgrade expenses?
- The results of the above steps need to be documented in form of SRS document, confirmed and signed-off by all parties which will be participating in the data integration project.

Implementation

Based on the BRS and SRS, a feasibility study should be performed to select the tools to implement the data integration system. Small companies and enterprises which are starting with data warehousing are faced with making a decision about the set of tools they will need to implement the solution. The larger enterprise or the enterprises which already have started other projects of data integration are in an easier position as they already have experience and can extend the existing system and exploit the existing knowledge to implement the system more effectively. There are cases, however, when using a new, better suited platform or technology makes a system more effective compared to staying with existing company standards. For example, finding a more suitable tool which provides better scaling for future growth/expansion, a solution that lowers the implementation/support cost, lowering the license costs, migrating the system to a new/modern platform, etc.

Testing

Along with the implementation, the proper testing is a must to ensure that the unified data are correct, complete and up-to-date.

Both technical IT and business needs to participate in the testing to ensure that the results are as expected/required. Therefore, the testing should incorporate at least Performance Stress test (PST), Technical Acceptance Testing (TAT) and User Acceptance Testing (UAT) PST, TAT (Technical Acceptance Testing), UAT (User Acceptance Testing).

Approaches Adding attributes

Create a new column in Pandas DataFrame based on the existing columns
While working with data in Pandas, we perform a vast array of operations on the data to get the data in the desired form. One of these operations could be that we want to create new columns in the DataFrame based on the result of some operations on the existing columns in the DataFrame. Let's discuss several ways in which we can do that.

Given a Dataframe containing data about an event, we would like to create a new column called *'Discounted_Price'*, which is calculated after applying a discount of 10% on the Ticket price.
Example 1: We can use **DataFrame.apply()** function to achieve this task.

```
# importing pandas as pd

import pandas as pd

# Creating the DataFrame
df = pd.DataFrame({'Date':['10/2/2011', '11/2/2011', '12/2/2011', '13/2/2011'],
                                    'Event':['Music', 'Poetry', 'Theatre', 'Comedy'],

                                    'Cost':[10000, 5000, 15000, 2000]})

# Print the dataframe
print(df)
```

	Date	Event	Cost
0	10/2/2011	Music	10000
1	11/2/2011	Poetry	5000
2	12/2/2011	Theatre	15000
3	13/2/2011	Comedy	2000

Now we will create a new column called 'Discounted_Price' after applying a 10% discount on the existing 'Cost' column.

```
# using apply function to create a new column
```

```
df['Discounted_Price'] = df.apply(lambda row: row.Cost - (row.Cost * 0.1),
axis = 1)

# Print the DataFrame after addition

# of new column

print(df)
```

	Date	Event	Cost	Discounted_Price
0	10/2/2011	Music	10000	9000.0
1	11/2/2011	Poetry	5000	4500.0
2	12/2/2011	Theatre	15000	13500.0
3	13/2/2011	Comedy	2000	1800.0

Now, we will create a mapping function (salary_stats) and use the DataFrame.map() function to create a new column from an existing column

```
def salary_stats(value):
     if value < 10000:
          return "very low"
     if 10000 <= value < 25000:
          return "low"
     elif 25000 <= value < 40000:
          return "average"
     elif 40000 <= value < 50000:
          return "better"
     elif value >= 50000:
          return "very good"

df['salary_stats'] = df['salary'].map(salary_stats)
display(df.head())
```

	name	salary	salary_stats
0	John	10000	low
1	Ted	20000	low
2	Dev	50000	very good
3	Brad	45500	better
4	Rex	19800	low

Approaches Adding data objects

Add one row in an existing Pandas DataFrame

Let us see how to add a new row of values to an existing dataframe. This can be used when we want to insert a new entry in our data that we might have missed adding earlier. There are different methods to achieve this. Now let's see with the help of examples how we can do this

Example 1:

We can add a single row using DataFrame.loc. We can add the row at the last in our dataframe. We can get the number of rows using **len(DataFrame.index)** for determining the position at which we need to add the new row.

```
import pandas as pd
from numpy.random import randint

mark = {'Name':['Harish', 'Rakshitha', 'Manohar', 'Vidya'],
           'Maths':[87, 98, 87, 95],
           'ITSkills':[83, 99, 84, 76]
      }

df = pd.DataFrame(mark)

display(df)
```

	Name	Maths	ITSkills
0	Harish	87	83
1	Rakshitha	98	99
2	Manohar	87	84
3	Vidya	95	76

```
df.loc[len(df.index)] = ['Hruthvik', 89, 96]

display(df)
```

	Name	Maths	ITSkills
0	Harish	87	83
1	Rakshitha	98	99
2	Manohar	87	84
3	Vidya	95	76
4	Hruthvik	89	96

Adding row/s to a file (Appending)

In this part we are going to discuss how to add/append row/s to a dataframe with multiple rows.

First import necessary libraries.

```
import pandas as pd
df=pd.read_csv('1.csv')
df
```

	Reg No	Name	M1	M2	M3
0	176CS21001	Anil	90	98	95
1	176CS21002	Sangamesh	95	93	96
2	176CS21003	Pooja	85	95	97
3	176CS21004	Jayashree	86	98	88

```
df1=pd.read_csv('2.csv')
df1
```

	Reg No	Name	M1	M2	M3
0	176CS21706	Vinod	92	99	95
1	176CS21707	Ravi	88	35	66

```
df=df.append(df1, ignore_index=True)
df
```

```
df
```

	Reg No	Name	M1	M2	M3
0	176CS21001	Anil	90	98	95
1	176CS21002	Sangamesh	95	93	96
2	176CS21003	Pooja	85	95	97
3	176CS21004	Jayashree	86	98	88
4	176CS21706	Vinod	92	99	95
5	176CS21707	Ravi	88	35	66

```
df3 = pd.concat([df, df1], ignore_index = True)
df3
```

```
df3 = pd.concat([df, df1], ignore_index = True)
```

```
df3
```

	Reg No	Name	M1	M2	M3
0	176CS21001	Anil	90	98	95
1	176CS21002	Sangamesh	95	93	96
2	176CS21003	Pooja	85	95	97
3	176CS21004	Jayashree	86	98	88
4	176CS21706	Vinod	92	99	95
5	176CS21707	Ravi	88	35	66
6	176CS21706	Vinod	92	99	95
7	176CS21707	Ravi	88	35	66